



# AI 서빙 전략 아이디어 V1

## 1. 현재 고민 포인트 1

배포 예산 및 환경

- 100만원 한도
- 후보: AWS g4dn (T4 GPU 16GB) vs RTX 4060 (VRAM 8GB)

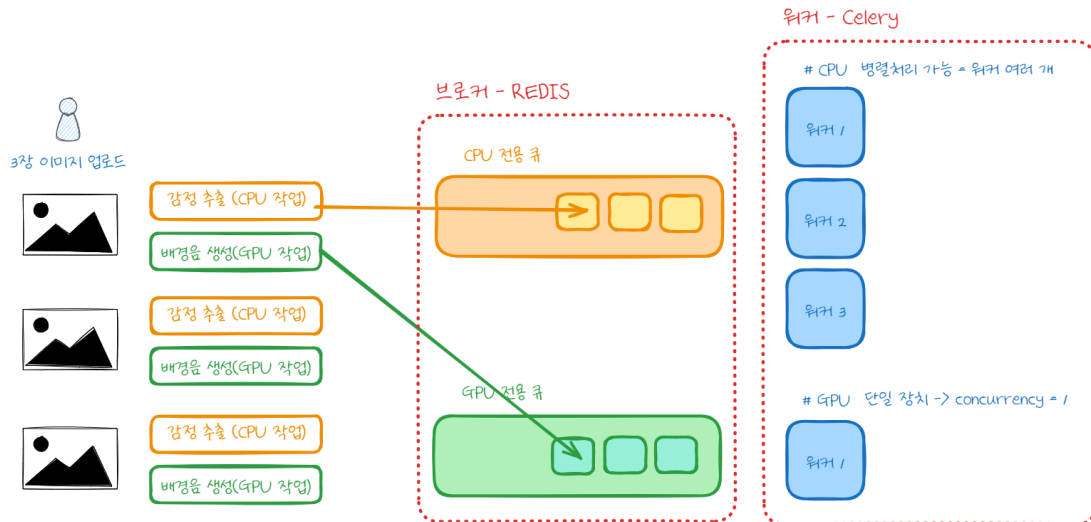
### **AWS g4dn.xlarge**

- **CPU:** 4 vCPU (Intel Xeon 기반, 클라우드 최적화)
- **RAM:** 16 GiB
- **GPU:** NVIDIA T4 (16GB VRAM, Turing 아키텍처)

### **데스크탑 (RTX 4060 장착 PC)**

- **CPU:** Intel i7-13620H / i7-13700H 계열 (추정, 13세대, 10코어~14코어 수준)
- **RAM:** **32 GB** (총 실제 메모리 32,444 MB 확인됨)
- **GPU:** RTX 4060 (8GB VRAM, Ada Lovelace 아키텍처)

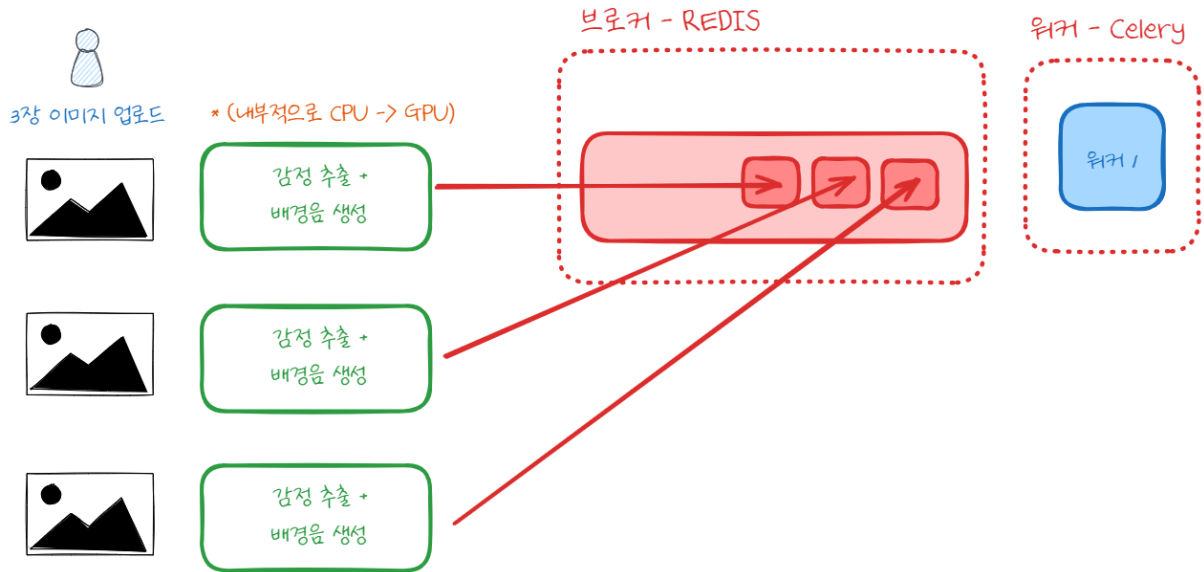
## 1. 처음 고민을 했던 비동기 구조



### [ TASK 분리 아이디어 ]

- 이미지 1장 처리 = 2개 TASK
  - CPU TASK: 감정 추출 (YOLO, CLIP, Moondream2 등 → 경량화 후 CPU로 실행)
  - GPU TASK: 배경음 생성 (MusicGen → GPU가 필수)
- CPU 전용 큐
  - 여러 워커 병렬 실행 가능 → 감정 추출을 빠르게 완료
- GPU 전용 큐
  - GPU 리소스는 단일 장치 → concurrency = 1 (한 번에 1개 작업만 처리)
  - CPU Task들이 동시에 끝나면 → GPU Task 대기열이 생기면서 병목 발생 가능
- 문제점
  - TASK가 세분화되어 큐 관리 복잡
  - GPU는 감정 추출이 끝나야 실행 가능 → 대기 시간 발생
  - 사용자 1명만 해도 6개 TASK라 → TASK가 너무 많아지는 단점이 있음

### 3. 궁금한 점 2



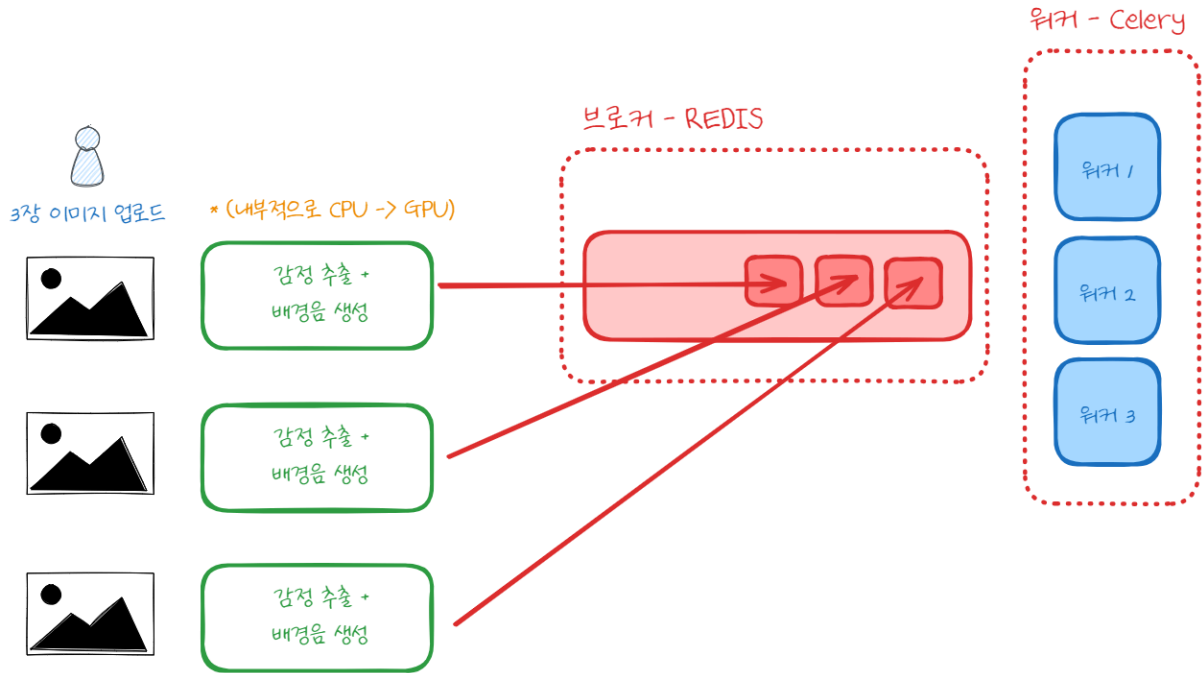
#### [ 이미지별로 TASK 생성 ]

- 감정 추출(CPU) + 배경음 생성(GPU)을 하나의 TASK 안에서 순차 실행
- 사용자 1명당 이미지가 3장이므로 이미지 단위로 CPU+GPU 순차 실행
- GPU는 1개이므로 → GPU 워커는 concurrency=1

#### 의문점?

현재 GPU는 1개이기 때문에 concurrency=1로 고정

- Task 1: CPU 감정 추출 → GPU MusicGen
- 이 Task가 CPU에서 돌아가는 동안 GPU는 아무 일도 안 함 (idle) → GPU idle 구간이 생김



그렇다면 워커 수를 여러 개를 두는 건 가능할까?

즉, 두 개 TASK가 동시에 GPU를 쓰면 OOM이나 커널 에러가 남  
GPU 구간은 직렬 실행이어야 함

- 만약 워커가 여러 개 있으면, 각자 CPU 구간을 먼저 실행
- CPU는 병렬 처리 가능하니까, 여러 워커가 동시에 감정 추출(CPU)을 돌릴 수 있음
- 그 다음 GPU 단계에 들어가려 하면 → "GPU 락(lock)"을 걸어 순차적으로만 실행.
- 이렇게 하면 CPU 구간은 병렬, GPU 구간은 직렬

#### 4. 궁금한 점 3

- 이미지 단위로 Task를 묶었다면, 굳이 CPU/GPU를 나눌 필요가 있을까?
- 가능하다면 모든 모델을 GPU에서 처리하는 게 단순하지 않을까?

BUT

- 메모리 문제(VRAM 한계)
  - 감정 추출 모델까지 GPU에서 돌리면 VRAM 사용량이 크게 증가
  - RTX 4060(8GB) 환경에서는 MusicGen + 다른 모델 동시 로딩 시 OOM 가능성 높음
  - AWS T4(16GB) 환경에서는 여유가 있지만, 그래도 안전성을 보장하기 어렵다
- 모델 특성 차이

- MusicGen → GPU 필수 (연산량 크고, CPU로는 사실상 불가능)
- 감정 추출(YOLO, CLIP, Moondream 등) → 경량화/양자화하면 CPU에서도 충분히 가능
- 리소스 활용 효율
  - CPU는 멀티코어 + RAM 32GB 등 자원이 넉넉
  - GPU는 단일 장치라 병목이 생기기 쉬움
  - 따라서 CPU와 GPU 역할을 분리하면 자원 활용이 더 균형적

## 5. 결론

- 모든 모델을 GPU에서 돌리는 건 단순하지만, VRAM 한계 때문에 위험하다.
- 감정 추출은 CPU로, MusicGen은 GPU로 분리하는 게 현실적으로 더 안정적이다.
- 즉, 각각의 성격(무거운 모델 vs 경량 모델)에 맞게 CPU/GPU를 나눠 쓰는 게 최적 전략.

### sequenceDiagram

```

participant User as 사용자
participant Queue as Task Queue (1개)
participant W1 as 워커1
participant W2 as 워커2
participant CPU as CPU 연산
participant GPU as GPU 연산 (concurrency=1)

```

User→>Queue: 이미지 단위 Task 2개 등록

Queue→>W1: Task1 할당

Queue→>W2: Task2 할당

W1→>CPU: 감정 추출 (병렬 실행)

W2→>CPU: 감정 추출 (병렬 실행)

CPU→>W1: 결과 전달

CPU→>W2: 결과 전달

W1→>GPU: MusicGen 실행 (10s)

Note right of GPU: GPU는 직렬 처리 중<br>W2는 대기

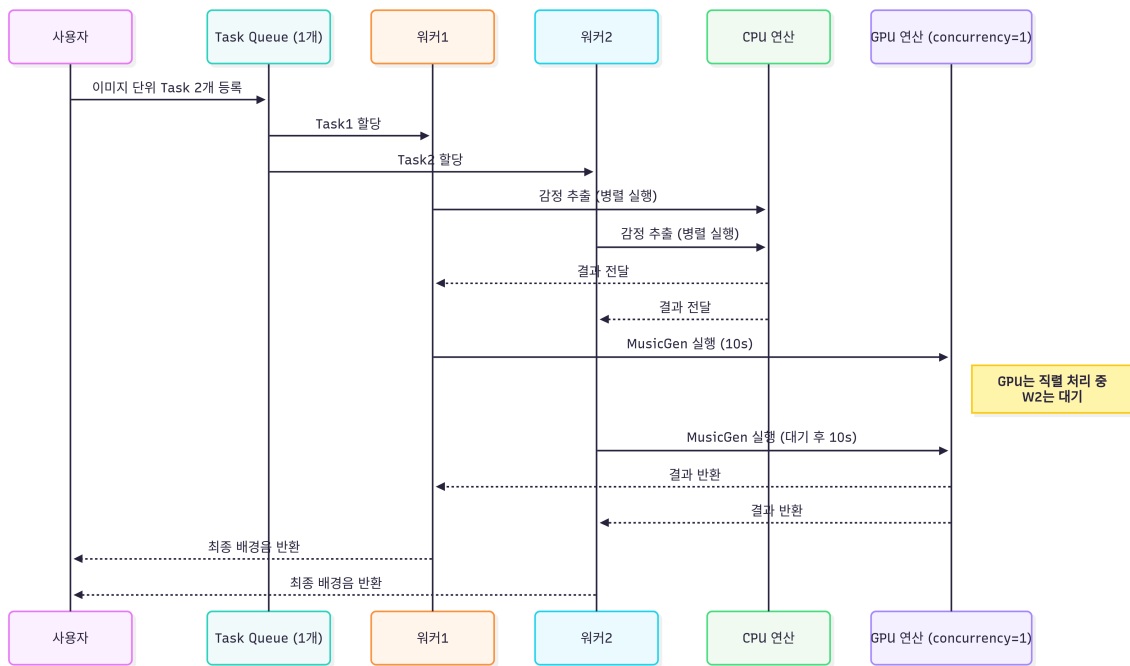
W2→>GPU: MusicGen 실행 (대기 후 10s)

GPU→>W1: 결과 반환

GPU→>W2: 결과 반환

W1→>User: 최종 배경음 반환

W2→>User: 최종 배경음 반환



## 추천 구조 (Hybrid: CPU + GPU 분리, 이미지 단위 Task)

### 1. Task 단위

- 이미지 단위 Task 생성
- 하나의 Task 안에서
  - Step1: 감정 추출(CPU)
  - Step2: 배경음 생성(GPU)
- 순차 실행 구조(Chain 활용)

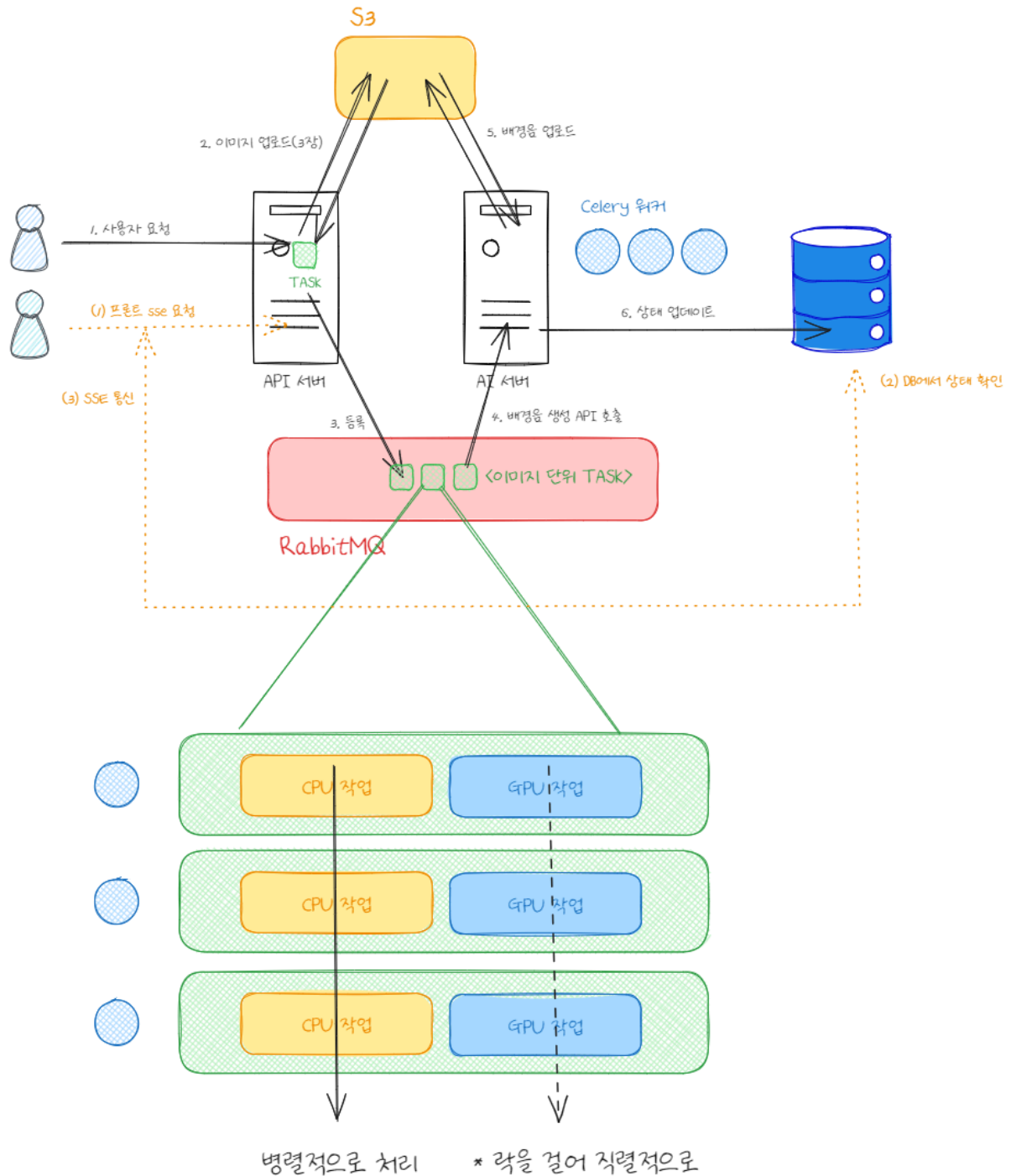
### 2. 큐 & 워커

- 큐: 하나
- 워커: 2~3개
  - CPU 구간은 병렬로 실행 → GPU idle 최소화
  - GPU 구간은 concurrency=1로 직렬 실행

### 3. 리소스 할당

- 감정 추출: CPU 전용 실행
  - YOLO → 경량화 버전 (ONNX, INT8)
  - CLIP, Moondream2 → 양자화/최적화
- 배경음 생성 (MusicGen): GPU 전용 실행
  - RTX 4060 → batch size 제한/FP16 활용
  - AWS T4 → VRAM 여유 있어 안정적 실행

그으으으라서 사용자 입장에서 부터 시스템 콜 플로우를 그려보자!



해야 할 일

- API랑 AI 서버의 통합의 유무
  - 오케스트레이션을 AI에서 할지 API에서 할지
- sse ping 15초(default) → 실험을 해보고 결정
  - HTTP 통신 네트워크 부담면에서
  - 그래서 이게 polling
  - 공부를 해서 → [WIKI에 정리](#)
- AI 서버 오케스트레이션
  - def def def def → def moondream2 / def musicgen / def 감정추출(CPU)
    - ~~generate music API~~
      - service
        - def moondream2 / def musicgen / def 감정추출(CPU)
        - GPU 세마포어로 걸기 기능 추가
          - 워커 수 4개(1개, 실험과 동반된 Trade-off)로 잡아서 CPU를 병렬적으로 돌리기 위해서
          - GPU 자원이 공유될 수 없는 임계 영역에서 돌아가야하는 자원이기 때문
    - 구현
      - 함수만 비워만 놓아라
      - log 넣고 / GPU 락까지 구현해
  - API 서버 왜 있는건데 → sse, get 목록, get 상세